

Resolução dos Exercícios

Capítulo: Funções e procedimentos

1. Indique se são verdadeiras ou falsas as seguintes afirmações:

- a) Uma função em C pode devolver simultaneamente mais do que um valor. - **falsa**
- b) Uma função em C pode não ter parâmetros. - **verdadeira**
- c) Uma função em C tem que devolver sempre um inteiro. - **falsa**
- d) Os parâmetros das funções podem ser do tipo **void**. - **falsa**
- e) A instrução **return** termina a execução de uma função. - **verdadeira**
- f) Uma variável local a uma função pode ter o mesmo nome que um parâmetro. - **falsa**
- g) A instrução **return** termina a execução de uma função apenas se for a última instrução da função em que se encontra. - **falsa**
- h) A instrução **return**, quando executada dentro de qualquer função, termina o programa. - **falsa**
- i) A instrução **return**, quando executada dentro da função **main**, termina o programa. - **verdadeira**
- j) O nome de uma função é opcional. - **falsa**
- k) Os parâmetros numa função são opcionais. - **verdadeira**
- l) Uma função deve fazer o maior número de tarefas possível sem ocupar muito código. - **falsa**
- m) Uma função não deve ter mais que 10 linhas - **falsa**
- n) O nome de uma função não deve ter mais do que 6 letras. - **falsa**
- o) O nome de uma função não pode ser uma palavra reservada do C. - **verdadeira**
- p) Sempre que for necessário devem ser utilizadas variáveis locais. - **verdadeira**
- q) Um protótipo não é nada mais que a repetição do cabeçalho da função seguido de; - **verdadeira**
- r) Em C, um procedimento não é mais do que uma função que "retorna **void**". - **falsa**

2. Identifique os erros de compilação que seriam detectados nos seguintes programas:

2.1:

```
/*
 * Copyright: Asneira Suprema Software!!!
 */

f(int x, int y);
{
    x = 4;
    y = 5;
}
```

O ponto e vírgula após o fechamento dos parênteses causará um erro de compilação. Isso ocorre porque `f(int x, int y);` passa a ser interpretado como um protótipo da função, fazendo com que o bloco de chaves `{}` subsequente fique 'solto' no código, sem estar vinculado a uma função.

2.2:

```
/*
 * Copyright: Asneira Suprema Software!!!
 */

void f(int x, int y)
{
    return -1;
}
```

A função está declarada com o tipo de retorno `void`, porém o seu corpo tenta retornar um valor inteiro (-1). Em C, funções `void` não devem retornar nenhum valor.

2.3:

```
/*
 * Copyright: Asneira Suprema Software!!!
 */

void f(void);
void f(int x, int y)
{
    x = 4;
    y = 5;
}
```

O protótipo da função declara que ela não recebe argumentos (`void`), porém a sua definição contém dois parâmetros inteiros `int x, int y`.

2.4:

```
/*
 * Copyright: Asneira Suprema Software!!!
 */

f (int x, int y);
void f(int x, int y)
{
    x = 4;
    y = 5;
}
```

Por padrão, funções em C que não são explicitamente tipadas assumem implicitamente o tipo `int` . No exemplo, a função `f` é declarada sem tipo (retornando `int` implicitamente), mas é definida com o tipo de retorno `void` , gerando um conflito de tipos de retorno.

2.5:

```
/*
 * Copyright: Asneira Suprema Software!!!
 */

void (int x, int y)
{
    x = 4;
    y = 5;
}
```

A função está sem nome. Em C, toda função precisa de um identificador.

2.6:

```
/*
 * Copyright: Asneira Suprema Software!!!
 */

void f(int x, y)
{
    x = 4;
    y = 5;
}
```

Todos os parâmetros da função devem ter seus tipos explicitamente definidos. No exemplo, o parâmetro `y` está sem tipo.

3. Exercício de exame

Dada as funções Ping e Pong

```
void Ping(int i)
{
    switch (i)
    {
```

```

        case 1:
        case 2:
        case 3: while (i--)
                    printf("\n%d", --i);
                    break;
        case 25: Pong(3);
                    break;
        default: printf("\nJá Passei em C");
                    Pong(123);
    }
}

```

```

void Pong(int x)
{
    int j=0;
    switch (x)
    {
        case 1:
        case 2: Ping(x);
        case 3: j=5;
                    j++;
                    return;
        default: printf("\nOla");
                    return;
    }
    printf("\nVou Sair");
}

```

Qual a saída das seguintes chamadas:

a) Pong(3);

■ Não imprime nada.

b) Ping(-4);

■ Já Passei em C
Ola

c) Ping(25);

┃ Não imprime nada.

d) Pong(2);

┃ 0

e) Pong(1);

┃ Entra num loop infinito e imprime os números negativos apartir de -1.

4. Exercício de exame

int abs(int x)

Devolve o valor absoluto de x.

abs(-5) --> 5

abs(5) --> 5

┃ [Ver código-fonte \(exe04.c\)](#)

5. Exercício de exame

float val(float x, int n, float t)

Devolve o val(valor atual líquido) para n anos, à taxa t e é definido através da seguinte fórmula

$$val = \frac{x}{1+t} + \frac{x}{(1+t)^2} + \frac{x}{(1+t)^3} + \dots + \frac{x}{(1+t)^n}$$

┃ [Ver código-fonte \(exe05.c\)](#)

6. Exercício de exame

long int n_segundos(int n_horas)

Devolve o número de segundos existentes em um conjunto de horas

n_segundos(0) --> 0

n_segundos(1) --> 3600

n_segundos(2) --> 7200

[Ver código-fonte \(exe06.c\)](#)

7. Exercício de exame

long int num(int n_horas, char tipo)

Semelhante à função anterior, só que recebe mais um parâmetro indicando aquilo que se quer saber 'h' - Horas, 'm' - Minutos e 's' - segundos.

num(3, 'h') --> 3

num(3, 'm') --> 180

num(3, 's') --> 10800

Resolva este exercício de três formas distintas: com a instrução **if-else**, **switch** com **break** e **switch** sem **break**

Nota: Supõe-se que o tipo está sempre correto.

[Ver código-fonte \(exe07.c\)](#)

8. Exercício de exame

float max(float x, float y, float w)

Devolve o maior dos valores x, y e w.

[Ver código-fonte \(exe08.c\)](#)

9. Exercício de exame

int impar(int x)

Devolve Verdade se x for ímpar. Falso, no caso contrário.

[Ver código-fonte \(exe09.c\)](#)

10. Exercício de exame

int entre(int x, int lim_inf, int lim_sup)

Verifica se x se encontra no intervalo **lim_inf <= x <= lim_sup**.

[Ver código-fonte \(exe10.c\)](#)

11. Escreva as seguintes funções

11.1

Função	Devolve
int isdigit(char c)	Verdade caso c seja dígito. Falso, caso contrário

[Ver código-fonte \(exe11-1.c\)](#)

11.2

Função	Devolve
int isalpha(char c)	Verdade caso c seja uma letra do alfabeto, maiúscula ou minúscula. Falso, caso contrário

[Ver código-fonte \(exe11-2.c\)](#)

11.3

Função	Devolve
int isalnum(char c)	Verdade caso c seja um dígito ou uma letra do alfabeto. Falso, caso contrário

[Ver código-fonte \(exe11-3.c\)](#)

11.4

Função	Devolve
int islower(char c)	Verdade caso c seja uma letra minúscula. Falso, caso contrário

[Ver código-fonte \(exe11-4.c\)](#)

11.5

Função	Devolve
int isupper(char c)	Verdade caso c seja uma letra maiúscula. Falso, caso contrário

[Ver código-fonte \(exe11-5.c\)](#)

11.6

Função	Devolve
int isspace(char c)	Verdade caso c seja um espaço ou um TAB. Falso, caso contrário

[Ver código-fonte \(exe11-6.c\)](#)

11.7

Função	Devolve
char tolower(char c)	Devolve o valor do caractere c transformado em minúsculas.

[Ver código-fonte \(exe11-7.c\)](#)

11.8

Função	Devolve
char toupper(char c)	Devolve o valor do caractere c transformado em maiúsculas.

[Ver código-fonte \(exe11-8.c\)](#)

12. Exercício de exame

`int is_square(int x, int y)`

Devolve um valor lógico que indica se x é ou não igual a y^2 .

[Ver código-fonte \(exe12.c\)](#)

13. Exercício de exame

`int minus(valor)`

Devolve o valor recebido sempre como número negativo.

`minus(10) --> -10`

`minus(-10) --> -10`

[Ver código-fonte \(exe13.c\)](#)

14. Exercício de exame

`int is_special(int x)`

Devolve um valor lógico que indica se o dobro de x é ou não igual a x^2

[Ver código-fonte \(exe14.c\)](#)

15. Exercício de exame

`int cubo(int x)`

Devolve o valor de x^3

[Ver código-fonte \(exe15.c\)](#)

16. Exercício de exame

`int isVogal(char c)`

Verifica se ch é uma das vogais do alfabeto (minúscula ou maiúscula)



[Ver código-fonte \(exe16.c\)](#)